

Experiment 1

Academic Year 2026-2027			
Program: Electronics Engineering (VLSI Design and Technology)			
Year of Study	Second Year	Semester	III
Course	Skill Lab: Python Programming	Course Code	VDCOR2VS201
Class	SY VLSI	Course In-Charge	Mr. Nirmol Munvar
Exp Name	Basic I/O operations and Dynamic Typing		
CO	Apply fundamental Python programming concepts such as variables, data types, operators, and input–output statements to develop basic programs for solving computational problems.		

Introduction to Python

Python is a high-level, general-purpose programming language known for its simple and readable syntax.

It is commonly used for:

- Web development
- Data science and AI
- Automation
- Desktop applications
- Scripting
- Machine learning
- Problem solving and competitive programming

A simple Python program:

```
print("Hello, World!")
```

Output:

Hello, World!

Why is Python beginner-friendly?

Python syntax is close to normal English. For example:

```
age = 20
```

```
print(age)
```

There is no need to explicitly declare that age is an integer.

Variables in Python

A **variable** is a name that refers to a value stored in memory.

```
name = "Rahul"
```

```
age = 20
```

```
marks = 85.5
```

Here:

- name → contains a string
- age → contains an integer
- marks → contains a floating-point number

Python determines the type automatically.

Dynamic Typing

Python is **dynamically typed**.

This means that you don't have to specify the data type of a variable when creating it.

For example:

```
x = 10
```

Python understands that x refers to an integer.

Later, you can assign a completely different type:

```
x = "Hello"
```

Now x refers to a string.

You can even do:

```
x = 10
```

```
print(x)
```

```
x = "Python"
```

```
print(x)
```

```
x = 3.14
```

```
print(x)
```

Output:

```
10
```

```
Python
```

```
3.14
```

Important idea

The **variable name does not permanently have a type**.

Instead, the **object/value that the variable refers to has a type**.

Think of it like this:

```
x —————> 10
```

integer

Then:

```
x —————> "Python"
```

string

So Python allows the same variable name to refer to objects of different types at different times.

How to Check the Type

Python provides the built-in `type()` function.

```
x = 10
```

```
print(type(x))
```

Output:

```
<class 'int'>
```

Another example:

```
name = "Alice"
```

```
print(type(name))
```

Output:

```
<class 'str'>
```

More examples:

```
a = 10
```

```
b = 3.14
```

```
c = "Hello"
```

```
d = True
```

```
print(type(a))
print(type(b))
print(type(c))
print(type(d))
```

Output:

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Common basic Python types

Type Example

```
int    10
float  10.5
str    "Hello"
bool   True / False
```

You can also check whether something is a particular type using `isinstance()`:

```
x = 10
```

```
print(isinstance(x, int))
```

Output:

```
True
```

What is an Interpreter?

An **interpreter** is a program that reads and executes your Python code.

For example, when you write:

```
print("Hello")
print(10 + 20)
```

the Python interpreter executes these instructions and produces:

```
Hello
```

```
30
```

A simplified view is:

Your Python code

↓

Python Interpreter

↓

Execution

↓

Output

Why do we need an interpreter?

Your computer's processor ultimately works with machine-level instructions. The Python interpreter acts as the software that understands Python code and manages its execution.

This is one reason Python is called an **interpreted language**, although modern Python implementations such as CPython also compile Python source into **bytecode** before executing it on the Python virtual machine.

For beginner-level understanding, remember:

Interpreter = program that executes Python code.

Input in Python

To take input from the user, Python uses:

```
input()
```

Example:

```
name = input("Enter your name: ")
```

```
print("Hello", name)
```

If the user enters:

Rahul

the output is:

Hello Rahul

Important: input() returns a string

Consider:

```
age = input("Enter your age: ")
```

Even if the user enters:

20

age is a **string**, not an integer.

You can verify:

```
print(type(age))
```

Output:

```
<class 'str'>
```

Converting Input to Numbers

If you want an integer:

```
age = int(input("Enter your age: "))
```

If you want a decimal number:

```
price = float(input("Enter the price: "))
```

For example:

```
a = int(input("Enter first number: "))
```

```
b = int(input("Enter second number: "))
```

```
print(a + b)
```

Input:

10

20

Output:

30

Basic Operators

Operators are symbols used to perform operations.

Arithmetic operators

Operator	Meaning	Example
+	Addition	10 + 3
-	Subtraction	10 - 3
*	Multiplication	10 * 3
/	Division	10 / 3
//	Floor division	10 // 3
%	Remainder	10 % 3
**	Power	10 ** 3

For example:

a = 10

b = 3

```
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

Output:

13

7

30

3.3333333333333335

3

1

1000

Basic Input/Output Exercise

Try solving this **without looking at the solution first**.

Exercise

Write a Python program that:

1. Takes the user's name.
2. Takes two integers.
3. Prints the user's name.
4. Prints the sum of the two numbers.
5. Prints their difference.
6. Prints their product.
7. Prints their division result.
8. Prints the remainder.
9. Prints the type of each number.

Example input

Enter your name: Rahul

Enter first number: 20

Enter second number: 6

Expected output

Hello Rahul

Sum: 26

Difference: 14

Product: 120

Division: 3.3333333333333335

Remainder: 2

<class 'int'>

<class 'int'>

Solution

```
name = input("Enter your name: ")
```

```
a = int(input("Enter first number: "))
```

```
b = int(input("Enter second number: "))
```

```
print("Hello", name)
```

```
print("Sum:", a + b)
```

```
print("Difference:", a - b)
```

```
print("Product:", a * b)
```

```
print("Division:", a / b)
```

```
print("Remainder:", a % b)
```

```
print(type(a))
```

```
print(type(b))
```

Quick concepts to remember

Python



High-level programming language

Variable



name = value

Dynamic typing



No need to declare variable type



```
x = 10
```

```
x = "Hello"
```

Check type



```
type(x)
```

User input



```
input()
```

Convert input



```
int(input())
```

```
float(input())
```

Output

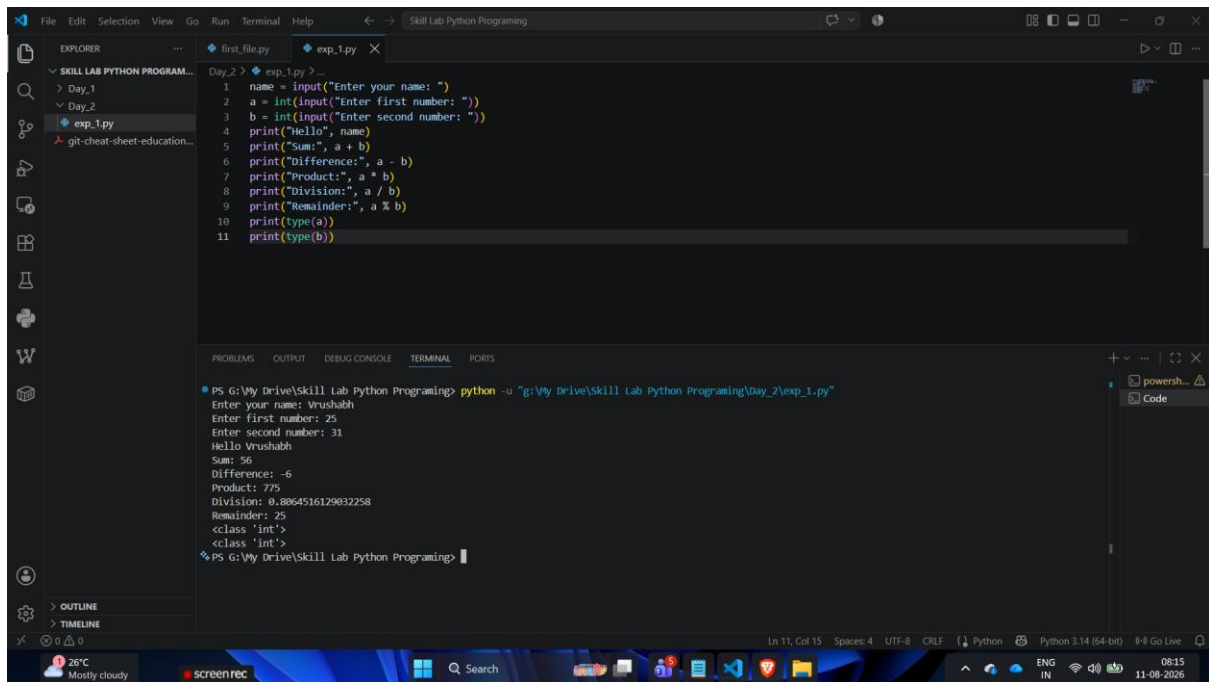


```
print()
```

Interpreter



Executes Python programs



The image shows a screenshot of a Visual Studio Code (VS Code) editor window. The editor is open to a file named `exp_1.py` in the `Day_2` folder. The code in the editor is as follows:

```
1 name = input("Enter your name: ")
2 a = int(input("Enter first number: "))
3 b = int(input("Enter second number: "))
4 print("Hello", name)
5 print("Sum:", a + b)
6 print("Difference:", a - b)
7 print("Product:", a * b)
8 print("Division:", a / b)
9 print("Remainder:", a % b)
10 print(type(a))
11 print(type(b))
```

The terminal window at the bottom shows the output of running the script using the command `python -u "G:\My Drive\Skill Lab Python Programming\Day_2\exp_1.py"`. The output is:

```
PS G:\My Drive\Skill Lab Python Programming> python -u "G:\My Drive\Skill Lab Python Programming\Day_2\exp_1.py"
Enter your name: Vrushabh
Enter first number: 25
Enter second number: 31
Hello Vrushabh
Sum: 56
Difference: -6
Product: 775
Division: 0.8064516129032258
Remainder: 25
<class 'int'>
<class 'int'>
PS G:\My Drive\Skill Lab Python Programming>
```

The status bar at the bottom indicates the current line and column (Ln 11, Col 15), the encoding (UTF-8), the line ending (CRLF), the language (Python), the Python version (Python 3.14 (64-bit)), and the Go Live status.